

230065H_a01

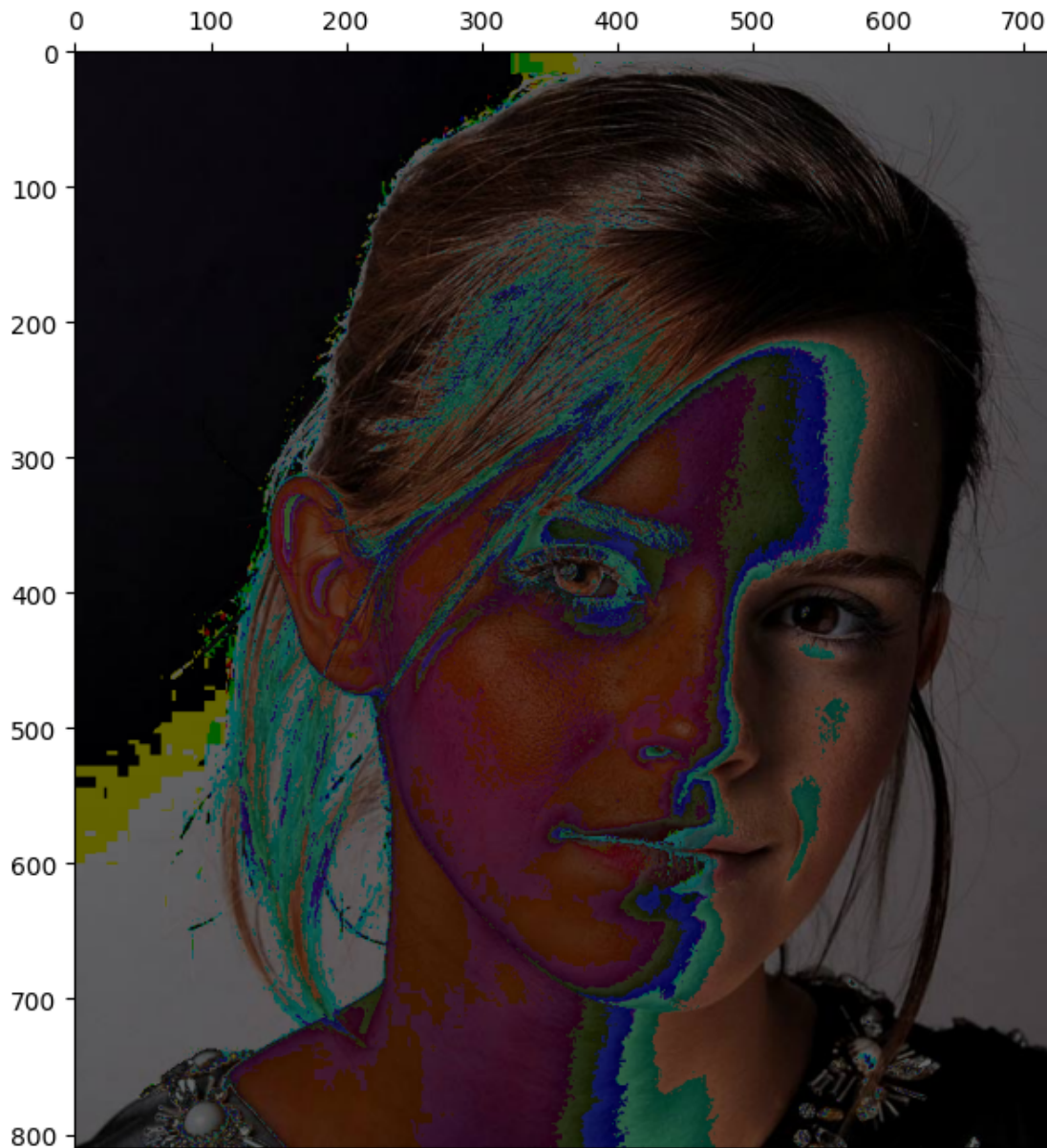
September 12, 2026

```
[74]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import convolve2d
```

```
[75]: f = cv.imread("images/emma.jpg")
# cv.namedWindow('Image', cv.WINDOW_AUTOSIZE)
# cv.imshow('Image', f)
# cv.waitKey(0)
# cv.destroyAllWindows()
```

```
[76]: fig, ax = plt.subplots(1,1,figsize=(8,8))
t1 = np.arange(100, dtype=np.uint8)
t2 = np.arange(50, dtype=np.uint8)
t3 = np.arange(106, dtype=np.uint8)
t = np.concatenate((t1,t2,t3))
print(t.shape)
f = t[f]
ax.imshow(cv.cvtColor(f, cv.COLOR_BGR2RGB))
ax.xaxis.set_ticks_position('top')
plt.show()
```

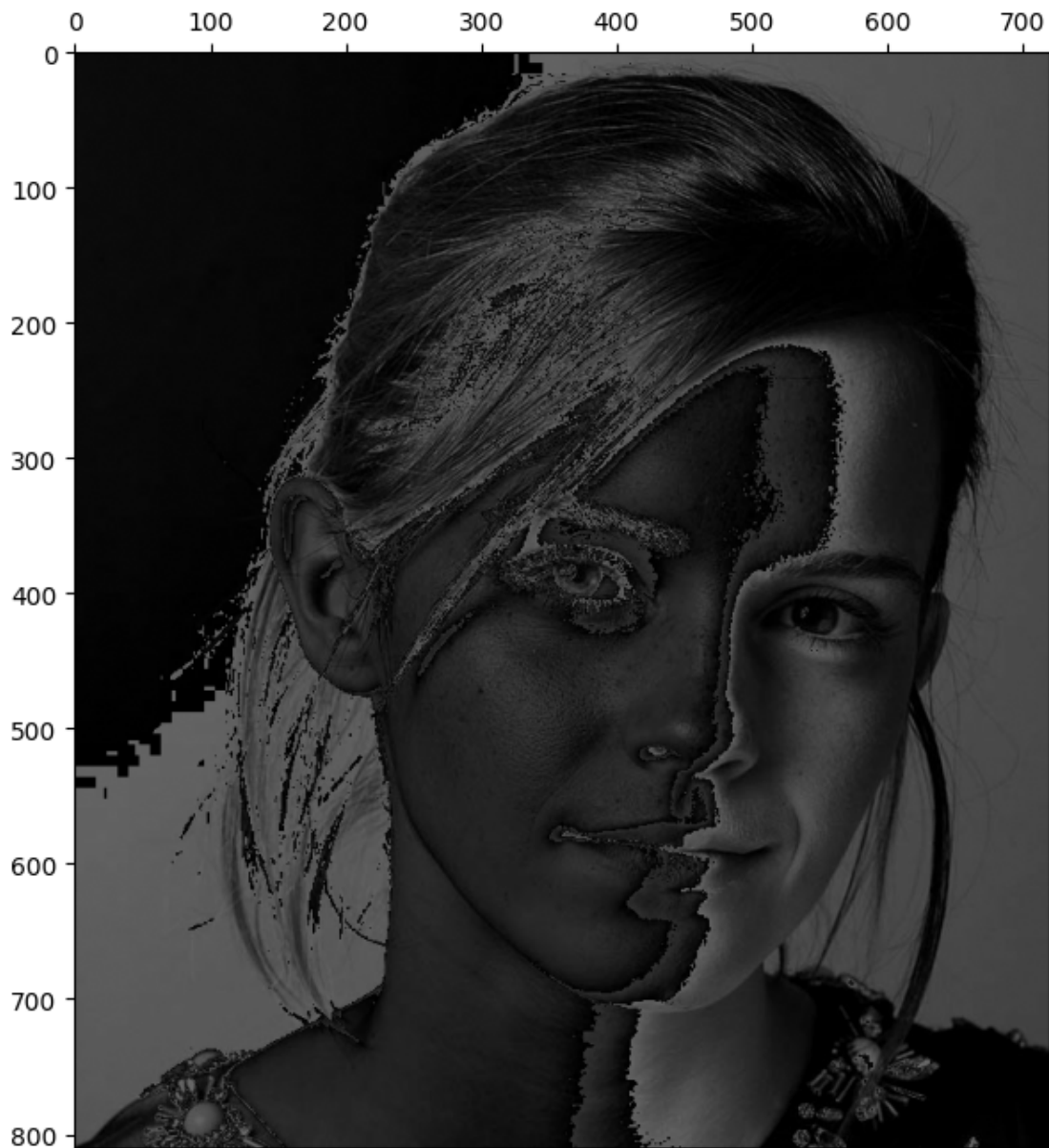
(256,)



```
[77]: f = cv.imread("images/emma.jpg", cv.IMREAD_GRAYSCALE)
fig, ax = plt.subplots(1,1,figsize=(8,8))
t1 = np.arange(100, dtype=np.uint8)
t2 = np.arange(50, dtype=np.uint8)
t3 = np.arange(106, dtype=np.uint8)
t = np.concatenate((t1,t2,t3))
print(t.shape)
f = t[f]
ax.imshow(f, cmap='gray', vmin=0, vmax=255)
ax.xaxis.set_ticks_position('top')
```

```
plt.show()
```

(256,)



```
[78]: def intensity_transform(im, breakpoints):  
    # 1. Unpack your breakpoints into two separate arrays: x_coords (input) and  
    ↪ y_coords (output).  
  
    # 2. Use np.interp() to evaluate the original image pixels against your  
    ↪ coordinates.
```

```

# Concept: np.interp(im, x_coords, y_coords)

# 3. The interpolation returns floats. Convert the array back to standard
→ image format using .astype(np.uint8).

# return the final_image
return

```

```

[79]: x = [0,50,50,100,150,150,255]
y = [0,50,100,170,255,150,255]
f = cv.imread("images/emma.jpg", cv.IMREAD_GRAYSCALE)
X = np.arange(256, dtype='uint8')
F = np.interp(X,x,y).astype(np.uint8)

print(F)
fig, axes = plt.subplots(1,3, figsize=(24,8))
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[1].imshow(F[f], cmap='gray', vmin=0, vmax=255)
axes[2].plot(X,F)
plt.show()

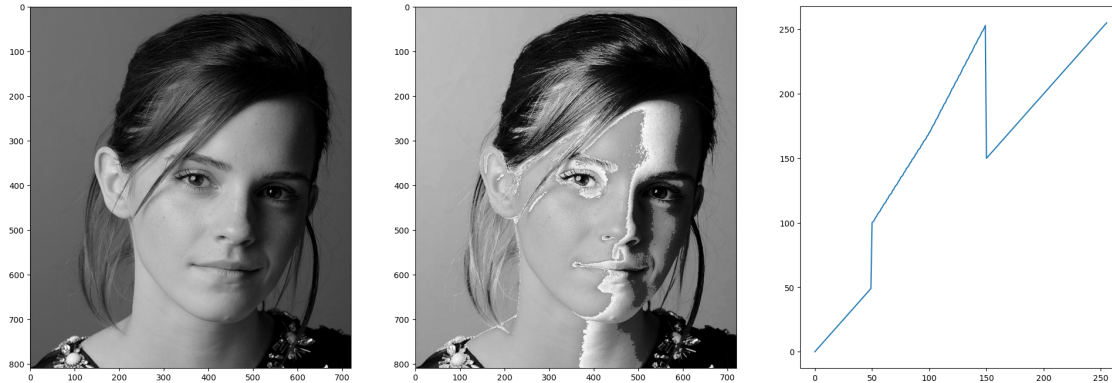
#print(X_interp)

```

```

[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17
 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35
 36 37 38 39 40 41 42 43 44 45 46 47 48 49 100 101 102 104
105 107 108 109 111 112 114 115 116 118 119 121 122 123 125 126 128 129
130 132 133 135 136 137 139 140 142 143 144 146 147 149 150 151 153 154
156 157 158 160 161 163 164 165 167 168 170 171 173 175 176 178 180 181
183 185 187 188 190 192 193 195 197 198 200 202 204 205 207 209 210 212
214 215 217 219 221 222 224 226 227 229 231 232 234 236 238 239 241 243
244 246 248 249 251 253 150 151 152 153 154 155 156 157 158 159 160 161
162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179
180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197
198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215
216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233
234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251
252 253 254 255]

```



```
[80]: def intensity_transform(im, breakpoints):
      x, y = zip(*breakpoints)
      X = np.arange(256, dtype='uint8')
      F = np.interp(X, x, y).astype(np.uint8)
      return F[im], X, F
```

```
[88]: f = cv.imread("images/emma.jpg", cv.IMREAD_GRAYSCALE)

# 3. Experiment with breakpoints (S-curve for pleasing contrast enhancement)
breakpoints = [(0, 0), (64, 32), (128, 128), (192, 224), (255, 255)]

# 4. Apply the transformation
transformed_img, X, F = intensity_transform(f, breakpoints)

# 5. Display the original image, transformed image, and plot side-by-side
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
# Move x-axis ticks to the top for all subplots

# Original Image
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original Image")
axes[0].xaxis.set_ticks_position('top') # Hides the axes ticks for a cleaner
↳ look

# Transformed Image
axes[1].imshow(transformed_img, cmap='gray', vmin=0, vmax=255)
axes[1].set_title("Transformed Image")
axes[1].xaxis.set_ticks_position('top') # Hides the axes ticks for a cleaner
↳ look

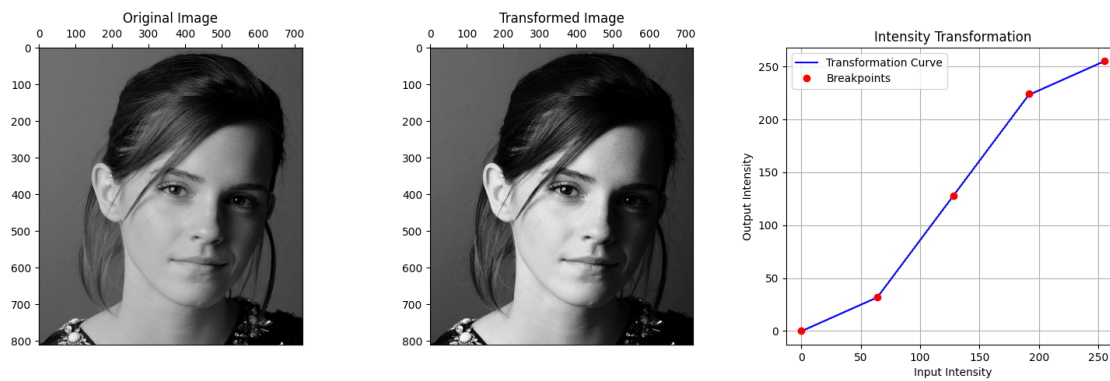
# Transformation Plot
axes[2].plot(X, F, color='blue', label='Transformation Curve')
# Optionally plot the actual breakpoints as red dots for clarity
```

```

x_coords, y_coords = zip(*breakpoints)
axes[2].plot(x_coords, y_coords, 'ro', label='Breakpoints')
axes[2].set_title("Intensity Transformation")
axes[2].set_xlabel("Input Intensity")
axes[2].set_ylabel("Output Intensity")
axes[2].grid(True)
axes[2].legend()

plt.tight_layout()
plt.show()

```



[]: